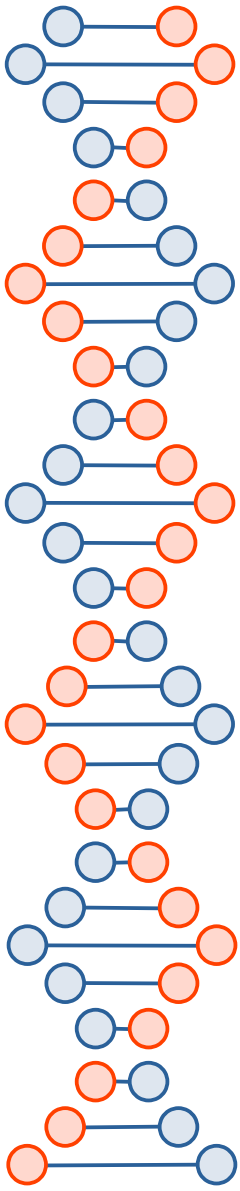


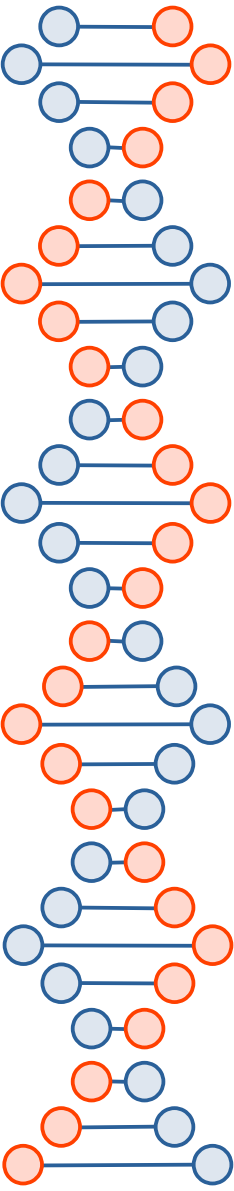
Métodos de testeo: El Hold-out y la Validación Cruzada

Dr. Josafá Pontes
Machine Learning



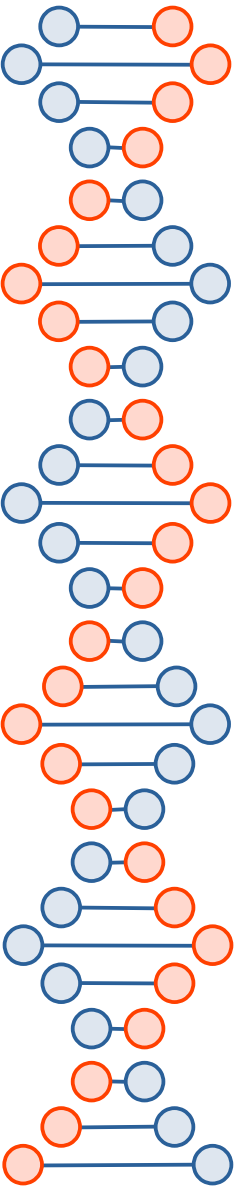
Outline

- Implement from scratch a train and test split of your data.
- Implement from scratch a k-fold cross-validation split of your data.



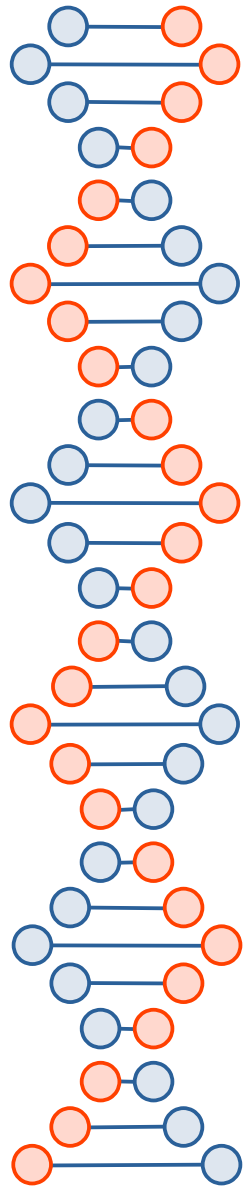
Resampling methods (1)

- The goal of predictive modeling is to create models that make good predictions on new data.
- We don't have access to this new data at the time of training, so we must use statistical methods to estimate the performance of a model on new data.
- This class of methods is called resampling methods, as they are resampling your available training data.
- The goal of resampling methods is to make the best use of your training data in order to accurately estimate the performance of a model on new unseen data.



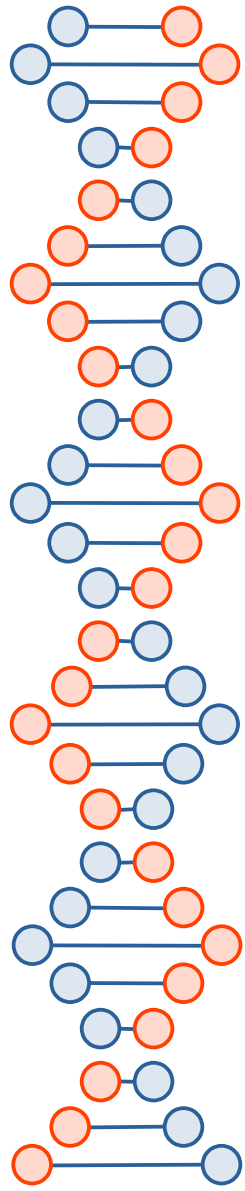
Resampling methods (2)

- Accurate estimates of performance can then be used to help you choose which set of model parameters to use or which model to select.
- Once you have chosen a model, you can train for final model on the entire training dataset and start using it to make predictions.
- There are two common resampling methods that you can use:
 - A train and test split of your data.
 - k-fold cross-validation.



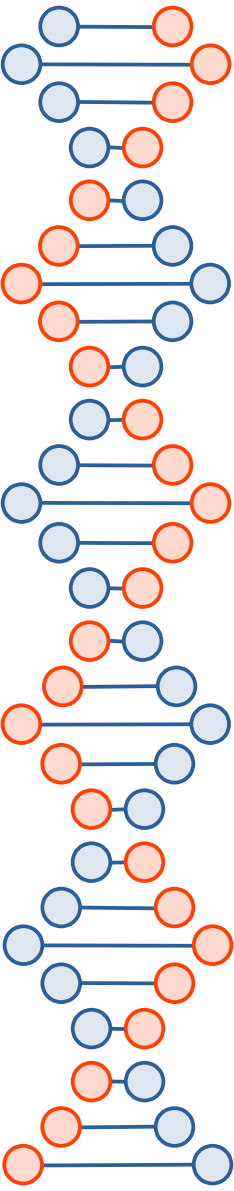
1. Hold-out method Train and Test Split (1)

- The train and test split is the easiest resampling method. As such, it is the most widely used.
- The train and test split involves separating a dataset into two parts:
 - 1. Training Dataset.
 - 2. Test Dataset.
- The training dataset is used by the machine learning algorithm to train the model.
- The test dataset is held back and is used to evaluate the performance of the model.
- The rows assigned to each dataset are randomly selected.
- This is an attempt to ensure that the training and evaluation of a model is objective.



1. Hold-out method Train and Test Split (2)

- If multiple algorithms are compared or multiple configurations of the same algorithm are compared, the same train and test split of the dataset should be used.
- This is to ensure that the comparison of performance is consistent or apples-to-apples.
- We can achieve this by seeding the random number generator the same way before splitting the data, or by holding the same split of the dataset for use by multiple algorithms.
- We can implement the train and test split of a dataset in a single function.

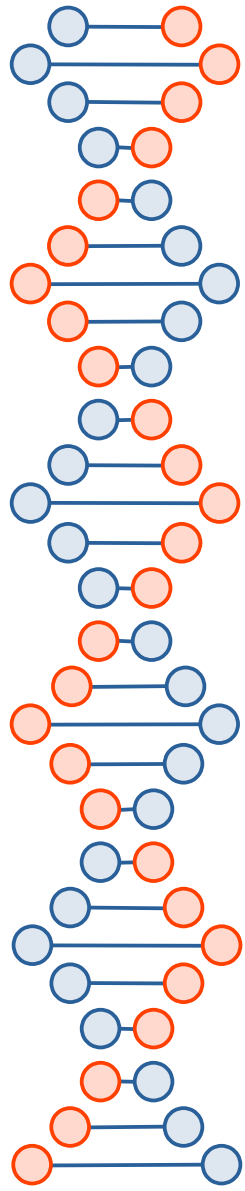


A train_test_split() algorithm (1)

- We define a function named `train_test_split()` to split a dataset into a train and test split.
- It accepts two arguments: the dataset to split as a list of lists and an optional split percentage.
- A default split percentage of 0.6 or 60% is used. This will assign 60% of the dataset to the training dataset and leave the remaining 40% to the test dataset. A 60/40 for train/test is a good default split of the data.
- The function first calculates how many rows the training set requires from the provided dataset.
- A copy of the original dataset is made.
- Random rows are selected and removed from the copied dataset and added to the train dataset until the train dataset contains the target number of rows.
- The rows that remain in the copy of the dataset are then returned as the test dataset.
- The `int(rand())` function from the random model is used to generate a random integer in the range between 0 and the size of the list.

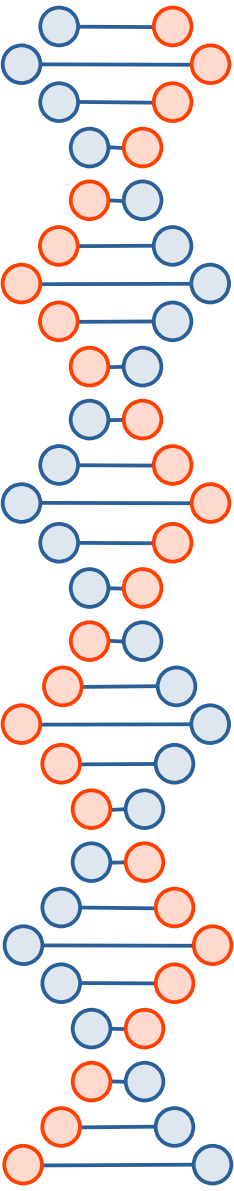
A train_test_split() algorithm (2)

```
sub train_test_split{  
  my ($self, $dataset, %args) = (splice (@_, 0, 2), split=>0.6, @_);  
  
  my $train_size = int($args{split} * @$dataset);  
  my @idx      = shuffle (0 .. $$dataset);  
  my @train_idx = @idx[0 .. $train_size -1];  
  my @test_idx  = @idx[$train_size .. $$dataset];  
  my @train     = @$dataset[@train_idx];  
  my @test      = @$dataset[@test_idx];  
  
  return \@train, \@test;  
}
```



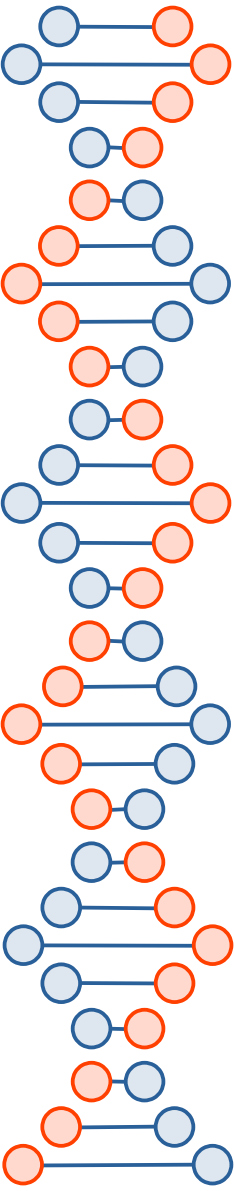
k-fold Cross-Validation Split (1)

- A limitation of using the train and test split method is that you get a noisy estimate of algorithm performance.
- The k-fold cross-validation method (also called just cross-validation) is a resampling method that provides a more accurate estimate of algorithm performance.
- It does this by first splitting the data into k groups.
- The algorithm is then trained and evaluated k times and the performance summarized by taking the mean performance score.



k-fold Cross-Validation Split (2)

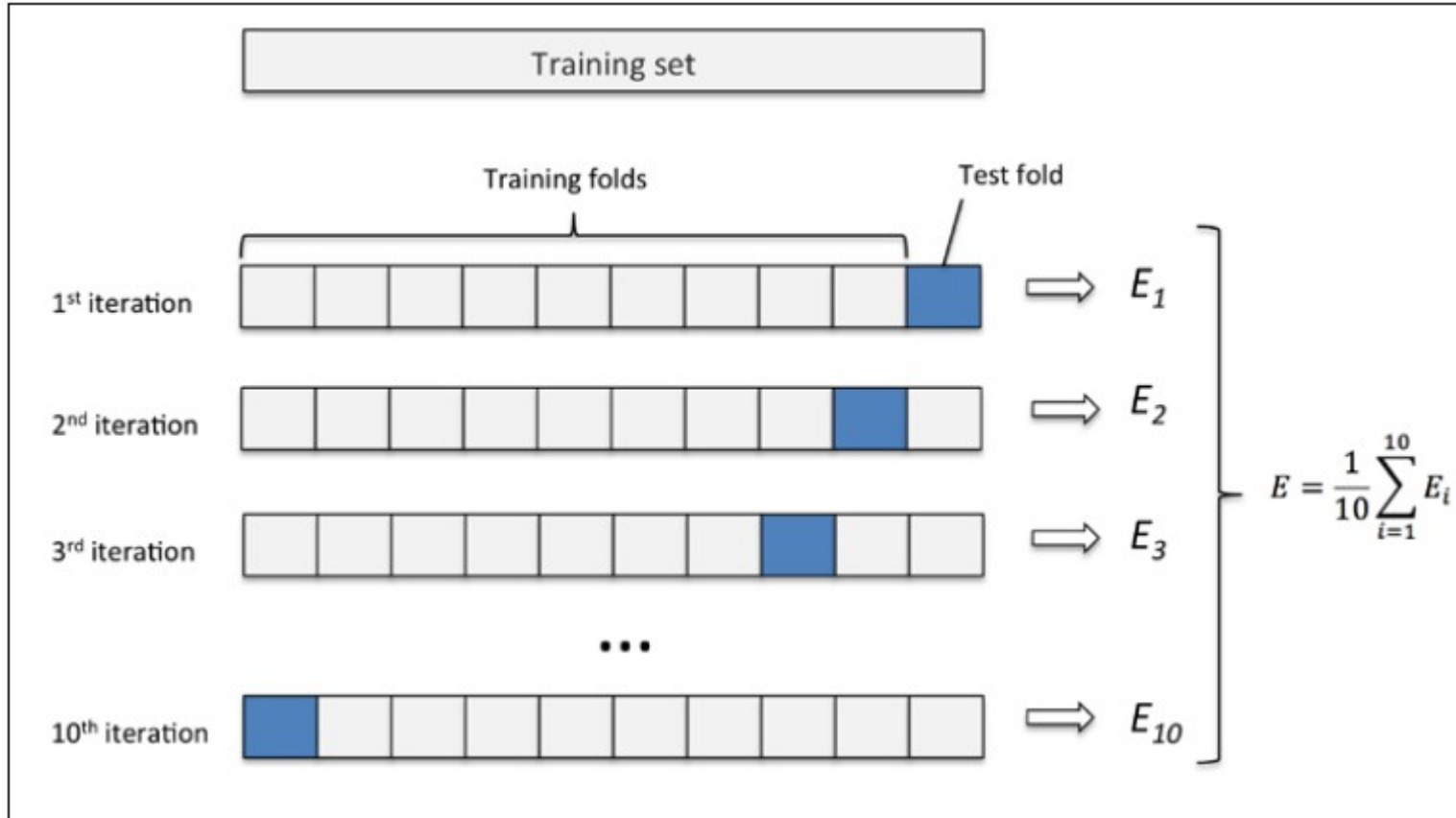
- Each group of data is called a fold, hence the name k-fold cross-validation.
- It works by first training the algorithm on the $k-1$ groups of the data and evaluating it on the k th hold-out group as the test set. This is repeated so that each of the k groups is given an opportunity to be held out and used as the test set.
- As such, the value of k should be divisible by the number of rows in your training dataset, to ensure each of the k groups has the same number of rows.

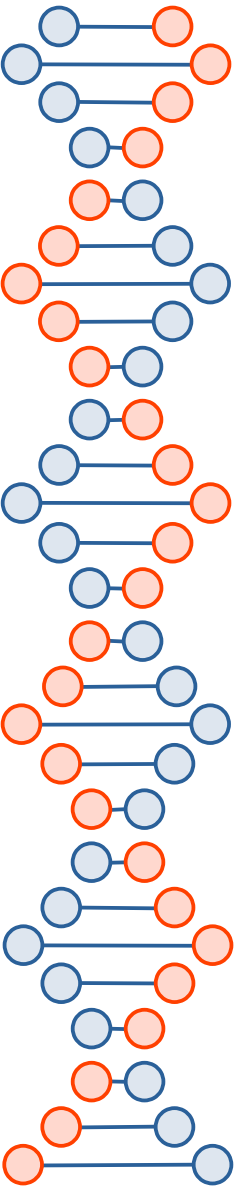


k-fold Cross-Validation Split (3)

- You should choose a value for k that splits the data into groups with enough rows that each group is still representative of the original dataset.
- A good default to use is $k=3$ for a small dataset or $k=10$ for a larger dataset.
- A quick way to check if the fold sizes are representative is to calculate summary statistics such as mean and standard deviation and see how much the values differ from the same statistics on the whole dataset.

k-fold Cross-Validation Split (4)





Cross validation split algorithm (2)

- We can reuse what we learned in the previous section in creating a train and test split here in implementing k-fold cross-validation. Instead of two groups, we must return k-folds or k groups of data.
- Below is a function named `cross_validation_split()` that implements the cross-validation split of data.
- As before, we create a copy of the dataset from which to draw randomly chosen rows.
- We calculate the size of each fold as the size of the dataset divided by the number of folds required.

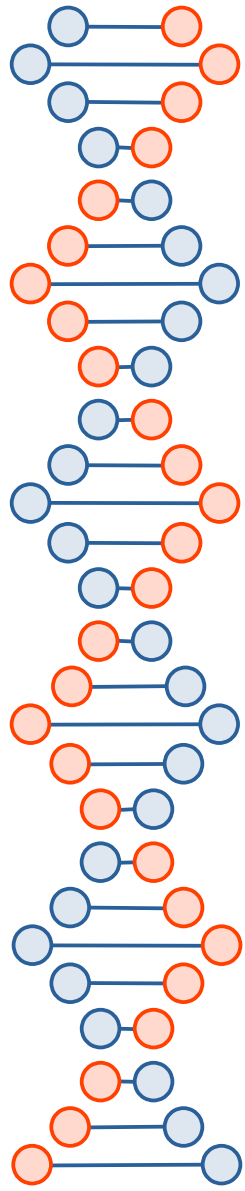
$$\text{fold size} = \text{count}(\text{rows}) / \text{count}(\text{folds})$$

- If the dataset does not cleanly divide by the number of folds, there may be some remainder rows and they will not be used in the split.
- We then create a list of rows with the required size and add them to a list of folds which is then returned at the end.

Cross validation split algorithm (2)

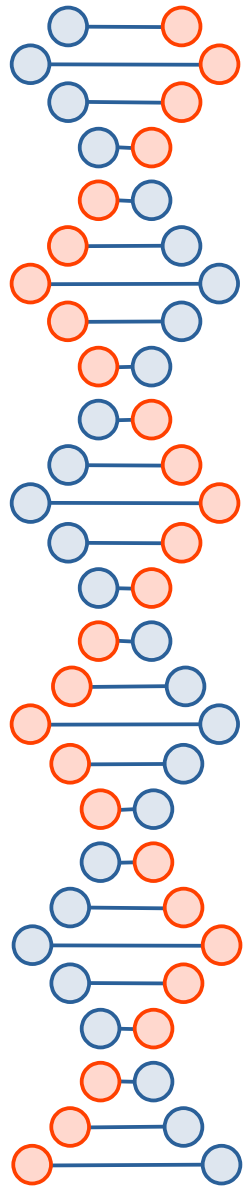
```
sub cross_validation_split{
  my ($self, $dataset, %args) = (splice (@_, 0, 2), n_folds=>10, @_);

  my @dataset_split;
  my $fold_size = int(@$dataset / $args{n_folds});
  my @idx       = shuffle (0 .. $$dataset);
  for my $i (0 .. $args{n_folds} -1){
    my @fold_idx = @idx[$i * $fold_size .. ($i +1) * $fold_size -1];
    push @dataset_split, [@$dataset[@fold_idx]];
  }
  return \@dataset_split;
}
```



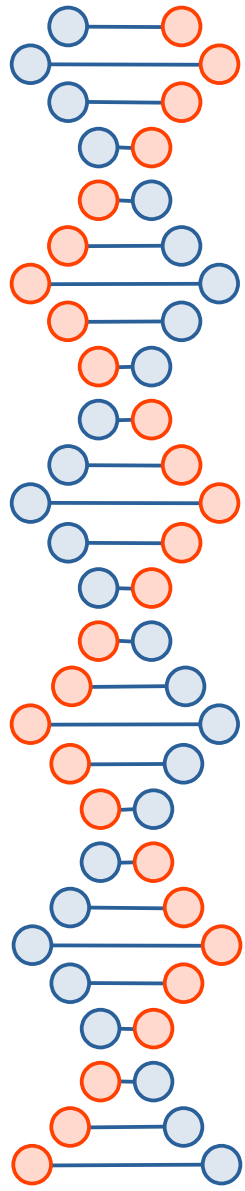
How to Choose a Resampling Method (1)

- The gold standard for estimating the performance of machine learning algorithms on new data is k-fold cross-validation.
- When well-configured, k-fold cross-validation gives a robust estimate of performance compared to other methods such as the train and test split.
- The downside of cross-validation is that it can be time-consuming to run, requiring k different models to be trained and evaluated.
- This is a problem if you have a very large dataset or if you are evaluating a model that takes a long time to train.



How to Choose a Resampling Method (2)

- The train and test split resampling method is the most widely used. This is because it is easy to understand and implement, and because it gives a quick estimate of algorithm performance.
- Only a single model is constructed and evaluated.
- Although the train and test split method can give a noisy or unreliable estimate of the performance of a model on new data, this becomes less of a problem if you have a very large dataset.
- Large datasets are those in the hundreds of thousands or millions of records, large enough that splitting it in half results in two datasets that have nearly equivalent statistical properties.
- In such cases, there may be little need to use k-fold cross-validation as an evaluation of the algorithm and a train and test split may be just as reliable.



Summary

- In this tutorial, you discovered how to implement resampling methods from scratch.
- Specifically, you learned:
 - How to implement the train and test split method.
 - How to implement the k-fold cross-validation method.
 - When to use each method.